Department of CSE (AIML), PES University
UE26CS151A -Python for Computational Problem Solving
Laboratory Week 4

Topics: Input/output, conditionals, loops, lists, strings, math

1) Easy(10M)- Count of matches in tournament

Problem Statement

You are given an integer n, the number of teams in a tournament that has strange rules:

- If the current number of teams is **even**, each team gets paired with another team. A total of n / 2 matches are played, and n / 2 teams advance to the next round.
- If the current number of teams is **odd**, one team randomly advances in the tournament, and the rest gets paired. A total of (n - 1) / 2 matches are played, and (n - 1) / 2 + 1 teams advance to the next round.

Return *the number of matches played in the tournament until a winner is decided.*

Conditions

- 1 <= n <= 200

Test Cases

Test Case 1

```
Input:
7

Output:
6
```

Test Case 2

```
Input:
14

Output:
13
```

Test Case 3

```
Input:
25

Output:
24
```

Test Case 4

```
Input:
189

Output:
188
```

Boilerplate Code

```
n = int(input())


matches = 0


- Solution -


print(matches)
```

Complete Solution

```
n = int(input())
matches = 0
while n > 1:
      if n % 2 == 0:
            matches += n // 2
            n = n // 2
      else:
            matches += (n - 1) // 2
            n = (n - 1) // 2 + 1
print(matches)
```

2) Medium(15 Marks) - Find k-length substrings with no repeated characters

Problem Statement

Given a string s and an integer k, return *the number of substrings in* s *of length* k *with no repeated characters*.

(Hint: There are only 26 english alphabets and O(26*n)~O(n)

Conditions

- 1 <= s.length <= 10^4
- s consists of lowercase English letters.

- $1 <= k <= 10^4$
- Input format = s k

Test Cases

Test Case 1

```
Input:
"havefunonleetcode" 5

Output:
6
```

Test Case 2

```
Input:
"home" 5


Output:
0
```

Boilerplate Code

```
s, k = input().split()

k = int(k)


# variables


while condition:


    if condition:

        # update
```

```
    else:

        # update

print(answer)
```

Complete Solution

```
s, k = input().split()
k = int(k)

count = 0
window = []

left = 0
right = 0

while right < len(s):

    # Add current character / handle duplicates
    while s[right] in window:
        window.remove(s[left])
        left += 1

    window.append(s[right])

    # Check window size
    if len(window) == k:
        count += 1
        window.remove(s[left])
        left += 1

    right += 1

print(count)
```

#**For the geeks and gods**: Instead of removing and adding from the window which can lead to simultaneous o(26) operations, use a bit mask of length 26 or an array of size 26 initialized with 0 and use left and right pointers along while tracking if array[index] where index here corresponds to the letter having ascii value of 97+index is 1 or not. This will result in constant operation.

3) Difficult(25 Marks) - Spiral Matrix

Problem Statement

Given an m x n matrix, return *all elements of the* matrix *in spiral order*.

Conditions

- m == matrix.length
- n == matrix[i].length
- 1 <= m, n <= 10
- -100 <= matrix[i][j] <= 100

Test Cases

Test Case 1

```
Input:
matrix = [[1,2,3],[4,5,6],[7,8,9]]
```

```
Output:
[1,2,3,6,9,8,7,4,5]
```

Test Case 2

Input:
[[1,2,3,4],[5,6,7,8],[9,10,11,12]]

Output:
[1,2,3,4,8,12,11,10,9,5,6,7]

Boilerplate Code

```
m, n = map(int, input().split())

matrix = []

for i in range(m):
    matrix.append(list(map(int, input().split())))

answer = []

top = 0
bottom = m - 1
left = 0
right = n - 1

while top <= bottom and left <= right:

    # top row
```

```
    # right column


    # bottom row


    # left column


print(answer)
```

Complete Solution

```
    m, n = map(int, input().split())


    matrix = []


    for i in range(m):
        matrix.append(list(map(int, input().split())))


    answer = []


    top = 0
    bottom = m - 1
    left = 0
    right = n - 1
```

```
while top <= bottom and left <= right:


    # top row
    for j in range(left, right + 1):
        answer.append(matrix[top][j])
    top += 1


    # right column
    for i in range(top, bottom + 1):
        answer.append(matrix[i][right])
    right -= 1
    # bottom row
    if top <= bottom:
        for j in range(right, left - 1, -1):
            answer.append(matrix[bottom][j])
        bottom -= 1
    # left column
    if left <= right:
        for i in range(bottom, top - 1, -1):
            answer.append(matrix[i][left])
        left += 1
print(answer)
```

Department of CSE (AIML), PES University
UE26CS151A -Python for Computational Problem Solving
Laboratory Week 4